C APPENDIX: COQ FORMALIZATION AND PROOF

We formalized our memory model in Coq and proved several key claims of this paper. The code is available from https://github.com/snu-sf/llvmtwin-coq. Note that we omit function calls and returns to simplify the formalization.

C.1 Definitions

The memory model is specified in file `Memory.v`. It differs in two ways from the presentation in this paper: (1) it does not support address spaces for brevity, and (2) memory maintains the last used block id, which is used to create fresh ids on allocation. The number of twin blocks ($|P|$) is set to 3.

Well-formedness of a memory block is defined in `Ir.MemBlock.wf`, and states, e.g., that $|P|$ is always 3, and the size of blocks is larger than zero. Well-formedness of memory is defined in `Ir.Memory.wf` and states, e.g., that all existing memory blocks are well-formed, and alive blocks have no overlapping addresses. Well-formedness of a state is defined in `Ir.Config.wf`, and includes assertions like the program counter is valid, memory is well-formed, etc. We proved that the execution of any instruction preserves the well-formedness of the input state.

The small-step semantics is defined in file`SmallStep.v`. Given an input state, a step can result in one of the following results: (1) success, if the execution of the following instruction was successful and yielded a new state, (2) UB, if the program raised undefined behavior, (3) OOM, if the program raised out-of-memory, and (4) halt, if the program finished.

Two states s and s' are twin with respect to a memory block l iff they are equal except for the configuration of l where the addresses of l in s' (P') are a permutation of those of s (P) and the enabled address is different (i.e., $P_0 \neq P'_0$).

A block's address is observed if a pointer to that block is given as argument to one of the following instructions: **ptrtoint**, **psub**, or **icmp**. Moreover, for the latter two instructions, the other argument must be a physical pointer. A pointer is guessed if it points to an unobserved block.

C.2 Proofs

We proved the following theorems in Coq:

Theorem C.1 (Twin allocation forbids pointer guessing). *Given two twin states s and s' w.r.t. l, where the next instruction dereferences a guessed pointer to l, the execution of either s or s' triggers UB.*

Two additional theorems are proved that establish the usefulness of twin states:

Theorem C.2 (Malloc). **malloc** *either returns a NULL pointer, or a logical pointer* $\mathsf{Log}(l, o)$. *Moreover, the states yielded by the small-step execution are all twin w.r.t. l.*

Theorem C.3. *Given two twin states s and s' w.r.t. l, the small-step execution of s and s' where the next instruction does not dereference a guessed pointer and does not observe block l either (1) halts, triggers UB or OOM, or (2) is successful and the successor states of s are twin with those of s'.*

We now show that certain instructions can be freely moved across (de)allocation functions:

Theorem C.4 (Instruction reordering). *Instructions* **icmp eq**, **icmp ule**, **psub**, **inttoptr**, **ptrtoint**, *and* **gep** *can be moved across* **malloc** *and* **free** *in both directions (upward and downward). Moreover, a program P' obtained from P by doing such reordering is equivalent to P.*

Finally we prove sufficient conditions for soundness of GVN:

Theorem C.5 (Soundness of GVN for pointers). *The four conditions given in Section 5 that state when it is sound to replace a pointer* `p` *with another pointer* `q` *are correct, i.e., the pointer replacement is a refinement.*